

```
Hello.csproj:
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>netcoreapp2.1</TargetFramework>
  </PropertyGroup>
</Project>
```

The project file specifies everything that's needed to restore dependencies and build the program. The *OutputType* tag specifies that we're building an executable — in other words, a console application. The *TargetFramework* tag specifies what .NET implementation we're targeting. In an advanced scenario, we can specify multiple target frameworks and build to all those in a single operation. In this tutorial, we'll stick to building only for .NET Core 1.0.

Program.cs

```
using System;
namespace Hello
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World from DotNET Core
Apps in Linux !");
        }
    }
}
```

 **Note:** Starting with .NET Core 2.0, we don't have to run 'dotnet restore' because it's run implicitly by all commands that require a restore to occur, such as 'dotnet new', 'dotnet build' and 'dotnet run'. It's still a valid command in certain scenarios in which doing an explicit restore makes sense, such as continuous integration builds in Azure DevOps Services or in build systems that need to explicitly control the time at which the restore occurs.

dotnet new calls *dotnet restore* implicitly. *dotnet restore* calls into NuGet (.NET package manager) to restore the tree of dependencies. NuGet analyses the *Hello.csproj* file, downloads the dependencies defined in the file (or grabs them from a cache on your machine) and writes the *obj/project.assets.json* file, which is necessary to compile and run the sample.



Important: If you're using a .NET Core 1.x version of the SDK, you'll have to call *dotnet restore* yourself after calling *dotnet new*.

The second command is:

```
$ dotnet run
```

dotnet run calls *dotnet build* to ensure that the build targets have been built, and then calls *dotnet <assembly.dll>* to run the target application.

```
$ dotnet run
```

```
Hello World from DotNET Core Apps in Linux!!
```

Alternatively, you can also execute *dotnet build* to compile the code without running the build console applications. This results in a compiled application as a DLL file that can be run with *dotnet bin\Debug\netcoreapp2.1\Hello.dll* on Windows (use / for non-Windows systems). You may also specify arguments to the application as you'll see later on.

```
$ dotnet bin\Debug\netcoreapp2.1\Hello.dll
Hello World!
```

As an advanced scenario, it's possible to build the application as a self-contained set of platform-specific files that can be deployed and run to a machine that doesn't necessarily have .NET Core installed.

Analyse your .NET apps for portability to .NET Core

Want to make your libraries multi-platform? Want to see how much work is required to make your application compatible with other .NET implementations and profiles, including .NET Core, .NET Standard, UWP, and Xamarin for iOS, Android and Mac? The .NET Portability Analyser is a tool that provides you with a detailed report on how flexible your program is across .NET implementations by analysing assemblies. The Portability Analyser is offered as a Visual Studio Extension and as a console app. You can get more details about it at <http://aka.ms/portabilityAnalyzer>.

In my next article, we will look at how to create a ASP.NET Core 2.1 Web application using Visual Studio code in Ubuntu, and run it in Linux containers. **END** 

References

- [1] <https://github.com/dotnet/core>
- [2] <https://www.dotnetconf.net/>
- [3] <https://www.techempower.com/benchmarks/#section=data-r16&hw=ph&test=plaintext>

Acknowledgement

My sincere thanks to Microsoft technical expert Sandeep Alur for reviewing this article

By: Maheshkumar Rajarathinavel

The author is a senior tech evangelist (ISV) in the One Commercial Partner team in India, and is based in Bengaluru. He works with numerous ISVs in India, and helps them architect and deploy their solutions on Microsoft Azure. You can reach him at maheshk@microsoft.com.